

SQL Languages

- DDL (Data Definition)
- DML (Data Manipulation)
- TCL (Transaction Control)
- DCL (Data Control)

Creating Database

CREATE DATABASE name-of-db;

Ex: CREATE DATABASE students;

Note: SQL is not case sensitive.

Creating table

```
CREATE TABLE student-info (  
  student-id int,  
  name varchar(255),  
  Address varchar(255),  
  City varchar(255),  
);
```

→ CREATE TABLE student-list AS
SELECT student-id, student-name
FROM student-info.

Insert Table

→ INSERT INTO table-name
VALUES (val1, val2, val3, ... val6)

* If we enter 6 columns then we have to add 6 columns must, or else there will be an error. Also, data types order also have to be maintained.

→ INSERT ALL FROM ORACLE
INTO mytable (c1, c2, ..., cn) VALUES (exp1, exp2, ..., expn)
INTO mytable (c1, c2, ..., cn) VALUES (exp1, exp2, ..., expn)

ALTER TABLE

* This is used to add, delete, or modify columns in an existing table.

Syntax:-

ALTER TABLE table-name } To add
ADD column-name datatype;

ALTER TABLE table-name } To remove
DROP COLUMN column-name;

ALTER TABLE table-name } To rename
RENAME new-table-name;

ALTER TABLE table-name
RENAME COLUMN column old-column-name
TO new-table-name

DROP, TRUNCATE & DELETE

	<u>DELETE</u> DML	<u>DROP</u> DDL	<u>TRUNCATE</u> DDL
Use	Deletes one or more existing records	Drops the complete table from database	Deletes all rows from the database
Syntax	DELETE FROM table-name WHERE condition;	DROP TABLE table-name;	TRUNCATE TABLE table-name;

~~Transitio~~

* We can get back the deleted items in the DELETE using ROLLBACK, But in the DROP & TRUNCATE we can't rollback

Constraints

NOT NULL, UNIQUE, PRIMARY & FOREIGN
KEYS, CHECK, DEFAULT, CREATE INDEX.

NOT NULL

* Constraints can be specified when the table is created with the CREATE TABLE statement, or after the table is created.

Ex:-
CREATE TABLE students (
student_id int NOT NULL;
name varchar(255) NOT NULL;
age int;
);

Ex: ALTER TABLE Students MODIFY age int
NOT NULL;

*MODIFY is used to add the Not Null constraint into the already created table.

Primary Key Constraint

1) Composite

a) MySQL

```
CREATE TABLE Students (  
  student_id int NOT NULL,  
  name varchar(255) NOT NULL,  
  age int,  
  PRIMARY KEY (id student_id)  
);
```

b) Oracle

```
CREATE TABLE Students (  
  student_id int NOT NULL PRIMARY KEY,  
  name varchar(255) NOT NULL,  
  age int,  
);
```

```
→ CREATE TABLE students (  
  student_id int NOT NULL,  
  name name varchar(255) NOT NULL,  
  age int,  
  CONSTRAINT pk_employees PRIMARY KEY  
  (id student_id, name)  
);
```


* We can compositely represent both the columns as primary keys using the constraint.

ADD
* ALTER TABLE students ADD
PRIMARY KEY (student_id);

→ Generally constraint name would be p.k - tablename

Note: - If you want to add a primary key to an already created table then you have to make sure the column should have no null values which you want to make the primary key

To DROP Primary Key
→ ALTER TABLE ~~students~~ students DROP PRIMARY KEY.

UNIQUE

* This ensures that all values in a column are different

* Both UNIQUE & PRIMARY KEY constraints provide a guarantee for uniqueness for a column or set of columns

My SQL

student_id int NOT NULL,
name varchar(255),

UNIQUE (student_id)

grade

student_id int NOT NULL UNIQUE,
name varchar(255) NOT NULL,

* UNIQUE & primary keys are almost
similar in terms of their usage

CHECK

Student_id int NOT NULL,

name varchar(255),

age int;

CHECK (age >= 0)

→ ALTER TABLE students

ADD CHECK (age >= 0);